Zenith: evaluating revisions to a reasoning graph

Will Cook • 12 October 2026

This is not financial advice. This paper describes experimental research. Historical examples and model outputs are not investment recommendations and must not be used to make financial decisions.

Abstract. Zenith studies whether recurring failures in a multi-stage language-model program can guide useful revisions. It records intermediate outputs, diagnoses errors and compares one revision with the original on later inputs. This edition prepares a mixed demonstration: real saved stock, ETF and prediction-market inputs, synthetic macro and news, and recomputed calculator summaries. The native graph loads the inputs and prepares its first reasoning request. No completed forecast or learned improvement is claimed. We also propose a controller that selects small prompt variations from context.

1. Evaluation protocol. Nodes read supplied evidence or earlier outputs and produce records for later nodes. **Lab** selects assets and forecasts from a saved information cutoff. **Oracle** compares the frozen forecast with later observations and proposes a diagnosis; **Evolve** proposes an instruction, context or dependency change. The base language model stays fixed during comparison.

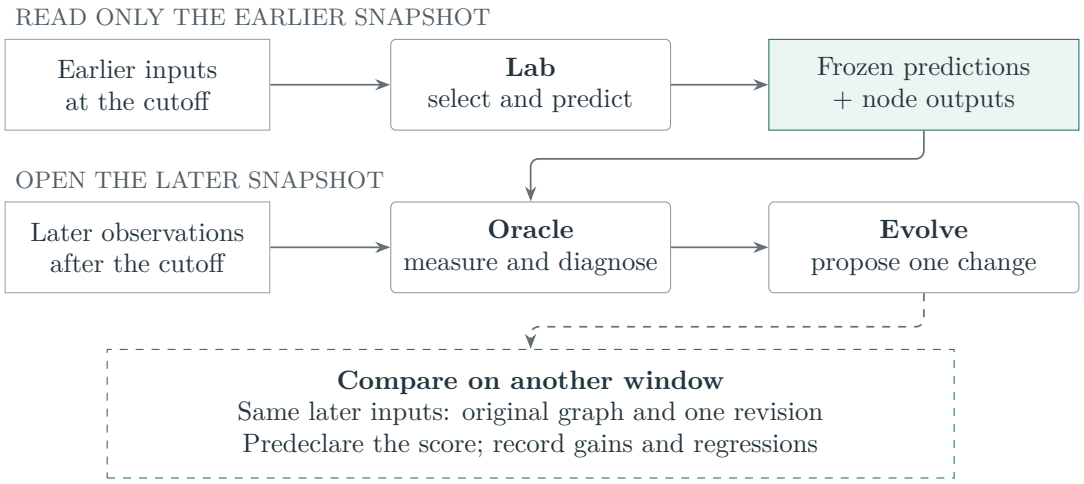


Figure 1. Proposed evaluation loop. Preserve the original record before revealing the outcome. The dashed step is a separate comparison, not evidence that a diagnosis or revision is correct.

Collect runs with the graph held fixed. Group failures across sliding windows and several widths, counting unique cases rather than overlapping windows. Freeze one revision and compare it with the baseline on matched later cases. Cases used to select a revision become development data for subsequent work.

Data and cutoffs. The demonstration keeps real stock, ETF and prediction-market inputs unchanged. Only macro and news are independently invented replacements; calculator summaries are recomputed from the actual mixed inputs. Per-feed hashes and labels identify this construction [1]. Real payloads remain private pending publication review. A packet can contain a current state or dated histories available at a cutoff. Predicting a later state and predicting a path need different targets and scores. Saved collection times do not verify every underlying availability time, and historical inputs cannot exclude later knowledge in model training.

Prior work. DSPy composes LM modules [5]; MIPRO optimises their instructions and examples against a downstream metric [6]. These are direct precedents for prompt optimisation. AlphaGo Zero [2] and autoresearch [3] motivated successive version comparisons. Zenith uses finite external histories; this prototype establishes no forecasting advantage.

2. The executable graph and failure diagnosis. The Lab has 25 nodes: six feed artifacts and 19 reasoning operators. Each miner is a feed-specific analysis step. Saved intermediate records let a reviewer locate an unsupported claim and inspect the evidence it received.

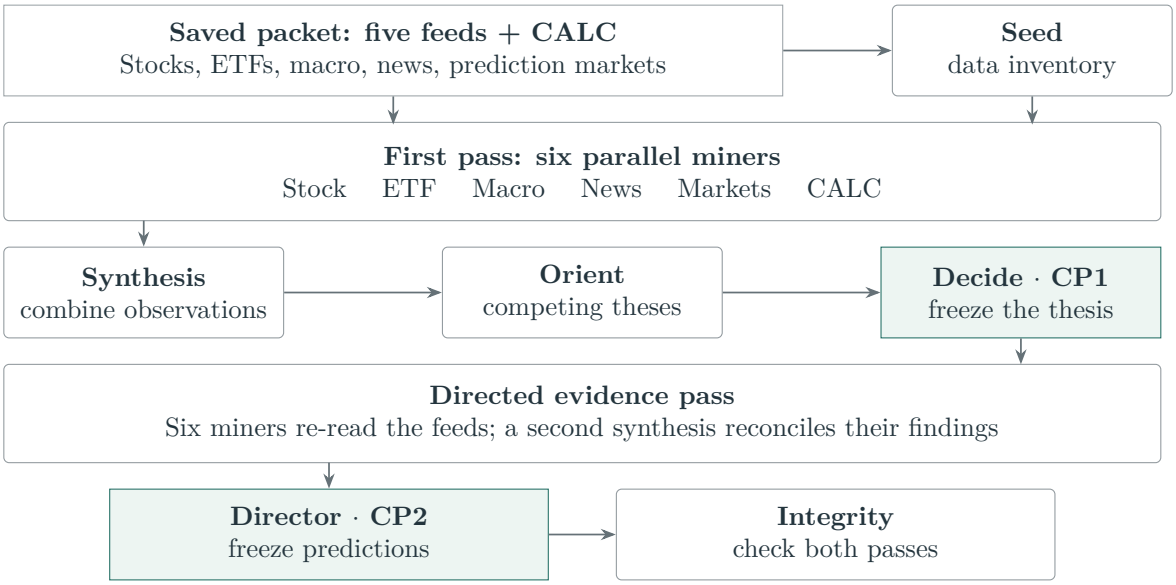


Figure 2. Lab's two passes. "Synthesis" denotes the nodes named Cross-correlation in the implementation; the second synthesis is grouped with the directed pass. CP1 is the saved thesis; in the quantile branch it also freezes forecast quantiles, which Director must preserve at CP2.

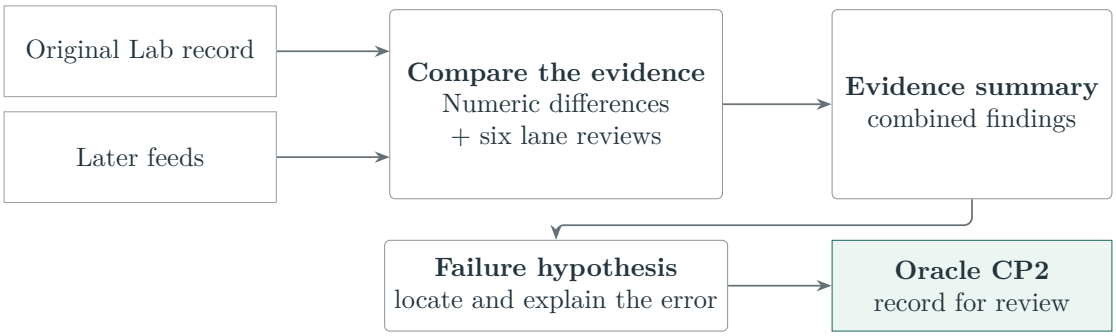


Figure 3. Oracle reviews the saved run against later evidence. Its attribution is a hypothesis about a failure, not a causal identification. Square boxes are records; rounded boxes are operations or groups. Exact feed, skip and cross-pass dependencies are retained in `docs/graphs.md`.

Diagnosis first asks what information was available:

Candidate failure	Question for the saved record
Evidence unavailable	Was the relevant information absent from the saved input universe?
Evidence not supplied	Was it in that universe but missing from the node's actual input?
Reasoning error	Did the node receive the evidence but draw an unsupported conclusion?
Execution failure	Did truncation, parsing or another runtime error prevent a usable output?

Keep unresolved cases unknown. Research performed after the outcome can help explain an event, but cannot become evidence the earlier run supposedly had. A repeated failure motivates an intervention test; frequency alone does not identify its cause.

3. Proposed context-conditioned prompts. Each reasoning node would keep a fixed task, input scope, output contract and work budget. A shared neural controller would read market-condition features available at the cutoff and select small wording variations for that node. We call its outputs *prompt settings*: numbers in $[0, 1]$ that index an ordered catalogue of text variants. They are distinct from the controller's learned numerical weights. This extension has not been implemented or tested.

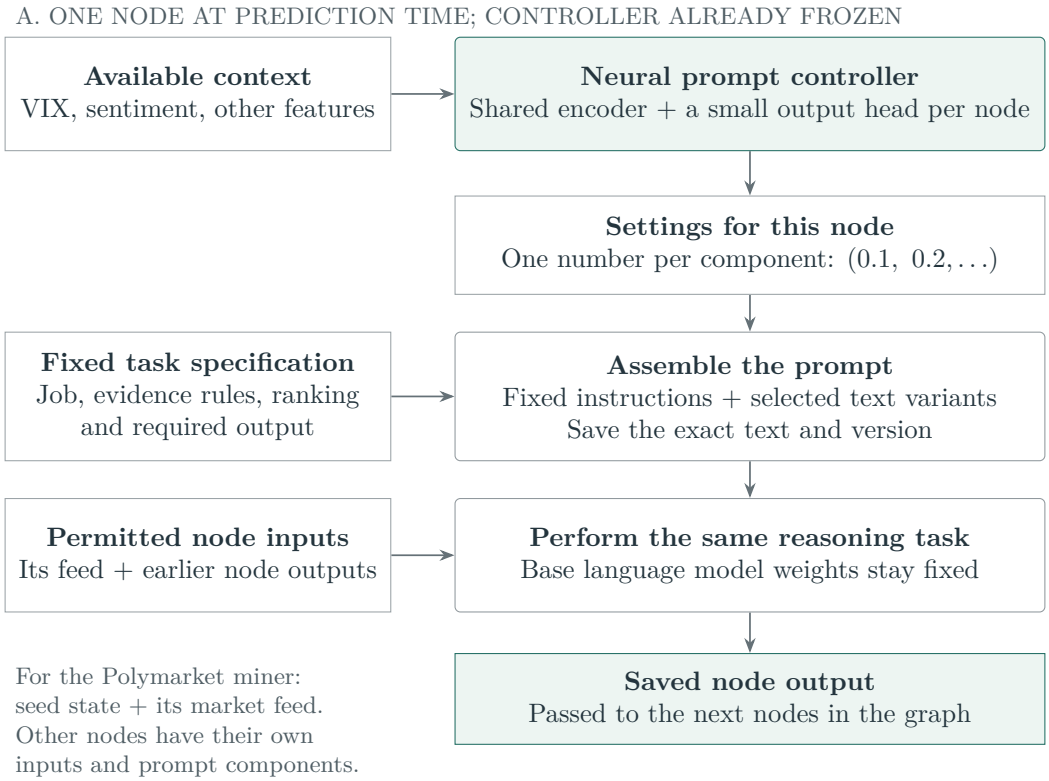
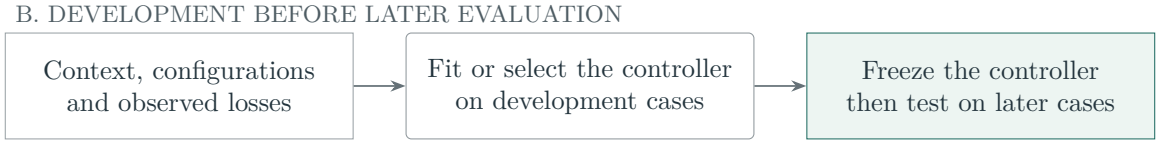


Figure 4. Settings select text; they do not multiply evidence or directly scale the model's reasoning. For example, 0.1 could select "consider sentiment as one explanation" and 0.2 "give sentiment slightly more interpretive weight where supported". Factual filters and ranking rules stay fixed. Nearby settings need not produce smoothly varying behaviour.



A first implementation could search a small catalogue, following discrete prompt optimisation [6]. A neural controller could instead learn from measured configuration losses. Backpropagation would train that controller; hard text selection and hosted LM calls provide no ordinary end-to-end derivative. ProTeGi [7] and TextGrad [8] use verbal feedback as a gradient analogy. Such feedback can suggest a revision, but is neither a numerical derivative nor proof of its benefit. VIX and sentiment are incomplete context features, not a complete regime classification. Later test outcomes must remain outside development.

4. What the demonstration establishes. The preparation command reads two saved input directories. It copies the real stock, ETF and prediction-market artifacts byte for byte, creates the labelled macro/news examples without reading their originals, and executes the existing calculator on the assembled inputs. The public manifest records source and input hashes; the public examples contain only the invented replacements [1].

Lane	Computation input	Public payload
Stocks, ETFs	Real saved tables	Withheld
Prediction markets	Real saved input	Withheld
Macro, news	Authored synthetic examples	Included
Calculator	Derived from mixed inputs	Withheld

Withholding a real payload from the website does not replace it in the actual computation. Synthetic examples are not anonymised observations and do not establish what happened in the historical market. The real-data source rights and permitted downstream uses remain under review; source-code availability alone does not settle those questions.

Verified boundary. The reduced graph passes topology validation. Native Lab execution loads all six artifacts and prepares the Seed request, where it awaits an agent response. The preparation uses zero model calls. It produces no completed CP1 or CP2, forecast, Oracle diagnosis or Evolve comparison. This is an executable input and handoff demonstration, not a financial backtest. Historical measurements from other configurations are not results for this one.

Next evaluation. First fix an authorised data and model setup, with reliable observation clocks. Complete Lab, then measure selection coverage, forecast error and cost under fixed rules. Use the same fallback for unselected assets and a predeclared score, such as quantile loss at fixed levels [4]. Compare one revision with the baseline on matched later cases. A selected subset can look accurate while missing useful targets; selection and conditional prediction error must both remain visible. Freeze the change before opening the comparison outcomes.

The proposed controller adds a further hypothesis: whether context-dependent prompt choices outperform fixed prompts. Test it on later cases, including regime changes, costs and regressions. A mixed-input demonstration cannot establish forecasting performance using real macro/news, and a finite later sample cannot establish reliability in every possible regime.

References

[1] Zenith. Mixed-input demonstration, per-feed provenance and preparation receipt, 2026.

[2] D. Silver et al. Mastering the game of Go without human knowledge. *Nature* 550:354–359, 2017. doi:10.1038/nature24270.

[3] A. Karpathy. *autoresearch*. Source inspected at `228791fb499a` (code).

[4] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *JASA* 102(477):359–378, 2007. doi:10.1198/016214506000001437.

[5] O. Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *ICLR*, 2024. arXiv:2310.03714.

[6] K. Opsahl-Ong et al. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. *EMNLP*, 2024. arXiv:2406.11695v2.

[7] R. Pryzant et al. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. *EMNLP*, 2023. arXiv:2305.03495.

[8] M. Yuksekgonul et al. TextGrad: Automatic "Differentiation" via Text. 2024. arXiv:2406.07496v1.